



LOST BOYS CYBER

THREAT INTELLIGENCE • MALWARE ANALYSIS • DETECTION ENGINEERING



Aviator Spider Purple Team exercise

Purple Team Exercise

Classification	TLP:CLEAR
Published	2026-06-24
Version	1.0
Author	Lost Boys Cyber
Report Type	Purple Team Exercise

TLP:CLEAR | Lost Boys Cyber | Aviator Spider Purple Team exercise - Purple Team Exercise

Published: 2026-06-24 | TLP:CLEAR | Version: 1.0 | Author: Lost Boys Cyber

AI Generation: This report was generated with AI assistance and is provided for informational purposes only. All findings, IOCs, detection rules, hunting queries, attribution assessments, and recommendations must be independently reviewed and validated by a qualified security professional before operational use.

Table of Contents

1. Executive Summary
2. Intelligence Requirement
3. Source Review and Confidence
4. Key Findings
 - 4.1. Threat findings
 - 4.2. Purple team exercise design summary
 - 4.3. MITRE ATT&CK mapping
 - 4.4. Exercise narrative
 - 4.5. Controls and constraints
 - 4.6. Detection coverage matrix
 - 4.7. Microsoft Defender KQL hunts
 - 4.8. Splunk SPL hunts
 - 4.9. Sigma detection logic
 - 4.10. YARA-style content triage for downloaded scripts
 - 4.11. Purple team injects and expected blue-team observations

1. Executive Summary

Lost Boys Cyber assesses with high confidence that AVIATOR SPIDER, also publicly tracked as TA2541 and Operation Layover, is a financially motivated cybercriminal activity cluster that has repeatedly targeted aviation, aerospace, transportation, manufacturing, and defense-adjacent organizations with high-volume phishing, cloud-hosted payload delivery, commodity remote access trojans, and crypter-enabled evasion. The actor is not assessed as technically elite; the operational risk comes from persistence, industry-specific lure tradecraft, reliable abuse of trusted hosting services, and a repeatable infection chain that can convert a single user execution event into remote access, credential exposure, surveillance, and follow-on fraud or intrusion enablement.

This report provides a purple team exercise package to emulate AVIATOR SPIDER in a controlled, defensible way. The exercise focuses on aviation-themed phishing, cloud-storage delivery, VBS/PowerShell execution, staged payload retrieval, persistence, security tool discovery, system discovery, process injection simulation, and RAT-like command-and-control telemetry. The design intentionally avoids deploying real malware or functional RATs; all payloads should be benign simulators, signed internal test binaries, Atomic Red Team-style commands, or telemetry-only scripts that create observable behaviors without credential theft, destructive action, or unauthorized external communication.

Field	Assessment
Report family	Threat Intelligence Report / Purple Team Exercise
Primary adversary label	AVIATOR SPIDER
Community identifiers	TA2541; Operation Layover
Actor motivation	Criminal / financial gain
Primary sectors	Aviation, aerospace, transportation, manufacturing, defense, travel-adjacent suppliers
Geographic assessment	Public reporting attributes actor activity to Nigeria / West Africa with high confidence for some infrastructure and persona pivots
Primary intrusion pattern	High-volume phishing using aviation/travel lures, trusted file hosting, VBS/PowerShell staging, commodity RAT payloads, and persistence
Exercise objective	Validate email, endpoint, identity, network, and hunt coverage against a TA2541-like intrusion chain
Safety constraint	Use benign emulation artifacts only; do not deploy real RATs, crypters, credential theft, or public C2 infrastructure
Overall confidence	High for actor TTP baseline; medium for current infrastructure specifics after early-2025 hosting changes

Key judgments:

- AVIATOR SPIDER is a suitable purple team emulation candidate for aviation and transportation defenders because its tradecraft is repeatable, publicly documented, and observable across multiple defensive control layers.
- The most important defensive control points are email/link detonation, cloud-hosted file download inspection, script interpreter controls, AMSI and PowerShell monitoring, startup-folder/run-key persistence detection, WMI/security-product discovery alerts, and outbound dynamic DNS or suspicious TLS C2 monitoring.
- The exercise should prioritize realistic business lures such as aircraft parts quotes, charter details, flight itineraries, fuel requests, cargo shipment documentation, and maintenance-provider communications rather than generic current-event phishes.

- AVIATOR SPIDER tradecraft overlaps with ordinary commodity malware campaigns; attribution should not be asserted from a single RAT or domain. The exercise should train analysts to cluster by lure theme, delivery chain, persistence, hosting patterns, and infrastructure keywords rather than by malware family alone.

2. Intelligence Requirement

The intelligence requirement is to translate public AVIATOR SPIDER / TA2541 reporting into a safe purple team exercise that measures whether a defender can prevent, detect, investigate, and contain a realistic aviation-themed commodity RAT intrusion chain.

Requirement	Exercise interpretation	Success criteria
Identify relevant adversary behavior	Map AVIATOR SPIDER TTPs into emulation steps	ATT&CK-mapped scenario covers initial access, execution, persistence, discovery, defense evasion, C2 staging, and containment
Validate preventive controls	Test email, web, endpoint, and script controls	Malicious-looking but benign lures are blocked, warned, isolated, or fully logged
Validate detection controls	Confirm SIEM/EDR/NDR coverage for VBS, PowerShell, cloud hosting, startup persistence, WMI, and DDNS	Alerts fire with enough context for triage and investigation
Validate threat hunting	Provide KQL/SPL hunts for the activity chain	Hunters can reconstruct execution lineage and suspicious network staging from telemetry
Validate response playbooks	Exercise triage, scoping, containment, and recovery	Responders isolate test host, disable persistence, preserve evidence, and identify likely exposed identities/assets
Improve aviation-specific readiness	Use sector-specific lures and supplier/travel workflows	Business teams and SOC analysts recognize realistic aviation-themed social engineering patterns

Scope assumptions for this exercise:

- The target environment is a Windows enterprise with Microsoft 365/Defender or equivalent EDR/SIEM telemetry, email security, DNS/proxy logs, and endpoint process telemetry.
- The exercise will use an internal lab domain or tenant-approved test infrastructure. No real third-party Google Drive, OneDrive, Discord, PasteText, ShareText, GitHub, Coudup, dynamic DNS, or public paste infrastructure should be abused during testing unless explicitly authorized and rate-limited by the client.
- Payloads will be inert and will not collect credentials, screenshots, keystrokes, browser cookies, files, or personal data.
- The goal is adversary emulation and defensive validation, not live phishing against unconsented employees. If user-facing phishing is required, handle it as a separate approved awareness exercise with HR/legal coordination.

3. Source Review and Confidence

Source	Relevance	Key reporting used	Confidence contribution
CrowdStrike adversary profile: AVIATOR SPIDER	Names actor, geography, motivation, identifiers, and recent hosting shift	High-volume opportunistic spam against aviation/aerospace; identifiers Operation Layover and TA2541; criminal financial motivation; Nigeria/West Africa; Google Drive usage	High for label, motivation, and current adversary-profile summary; medium for details truncated behind vendor profile

		beginning in 2021; early-2025 shift toward Coudup hosting; use of Fsociety, Roda Crypt, and ScrubCrypt crypters	
MITRE ATT&CK G1018: TA2541	Technique and software mapping	TA2541 targets aviation/aerospace/transportation/manufacturing/defense since at least 2017; high-volume campaigns; commodity RATs obfuscated by crypters; techniques include domains/web services, VBS/PowerShell, persistence, AMSI/security tool modification, dynamic DNS, encrypted C2, process injection, WMI, and user execution	High for ATT&CK mapping and defensible emulation plan
Proofpoint: Charting TA2541's Flight	Detailed campaign chain and IOCs	Phishing with aviation/travel themes; macro attachment history; pivot to cloud-hosted payloads such as Google Drive; VBS/PowerShell staging; Pastetext/ShareText/GitHub staging; AsyncRAT preference with NetWire, WSH RAT, Parallax, Agent Tesla, Imminent Monitor and others; infrastructure keywords kimjoy, h0pe, grace; sample persistence examples	High for delivery chain, malware families, and purple team telemetry design
Cisco Talos: Operation Layover	Actor profiling and OSINT pivots	Aviation targeting for at least two years by 2021; broader malware operation active for years; Nigeria-based assessment; off-the-shelf malware; crypter use; AsyncRAT/njRAT; Google Drive-linked VBS; dynamic DNS domains and config identifiers; caution around weak infrastructure links	High for actor tradecraft and confidence discipline
Security community reporting / ET signatures	Detection context	Emerging Threats signatures for paste.ee, PowerShell decimal encoded RUNPE, and Google Drive double-extension VBS/PIF downloads	Medium; useful for detection ideas but should be validated against local sensor coverage

Analytic confidence:

- High confidence: AVIATOR SPIDER / TA2541 uses aviation-themed high-volume phishing, trusted hosting, VBS/PowerShell staging, commodity RATs, and persistence via startup folder, run keys, or scheduled tasks.
- High confidence: aviation, aerospace, transportation, manufacturing, and defense organizations are recurring targets.
- Medium confidence: early-2025 hosting preference changed from Google Drive to Coudup. CrowdStrike reports this shift, but public technical details are limited in the accessible profile.

- Medium confidence: historical IOCs remain useful for training and retro-hunting but should not be treated as current block-only indicators without age, sinkhole, or reassignment review.
- Low confidence: attribution of any single new incident to AVIATOR SPIDER based only on one RAT family, DDNS domain, or cloud-hosting provider. Those are common in commodity malware ecosystems.

4. Key Findings

4.1. Threat findings

Finding	Confidence	Defensive implication
AVIATOR SPIDER relies on repeatable social engineering rather than bespoke exploitation	High	User execution, email link inspection, file detonation, and script execution controls are primary prevention points
The actor's lure language is unusually stable and aviation-specific	High	Detection should include sector-specific lure terms such as flight, aircraft, charter, fuel, cargo, itinerary, Bombardier, yacht, quote, parts, and PPE cargo variants
Trusted cloud and web services are central to delivery and staging	High	Proxy, CASB, DNS, browser isolation, and file download telemetry must retain full URL, referrer, MIME type, and file extension context
VBS and PowerShell remain high-value telemetry points	High	Script block logging, AMSI, command-line capture, and parent/child process trees are essential
Commodity RAT payloads create broad but non-unique signals	High	Detection should emphasize behavior chain and configuration artifacts, not malware family name alone
Persistence frequently uses startup folders, registry run keys, and scheduled tasks	High	Endpoint detections should prioritize user-profile startup paths and HKCU Run entries created by script interpreters or suspicious temp paths
Dynamic DNS and low-cost infrastructure recur in historical reporting	Medium-high	Monitor DDNS domains and suspicious TLS SNI, but apply age and context before blocking old indicators
The actor's sophistication is moderate, but campaign longevity increases risk	High	Purple team success should be measured by early disruption and rapid scoping, not only by malware prevention

4.2. Purple team exercise design summary

Phase	Emulated behavior	Benign exercise artifact	Primary telemetry
Recon / preparation	Register lookalike test domain and stage benign payload	Internal lab domain such as aviation-docs.test; harmless text or signed test EXE	DNS, proxy, certificate logs
Initial access	Aviation-themed email with link to hosted double-extension script	Approved test email linking to `Charter_Details.pdf.vbs` or equivalent inert file	Email gateway, URL defense, user click telemetry
Execution	User opens benign VBS	VBS writes marker file and	DeviceProcessEvents, EDR

	launcher	launches PowerShell with safe parameters	process tree, AMSI
Staging	PowerShell retrieves benign payload from internal web/paste simulator	Local web server returns harmless payload or canary text	Proxy, PowerShell logs, network events
Persistence	Create removable persistence marker	Startup-folder VBS and scheduled task with clear exercise naming	Registry, scheduled task, file events
Discovery	Query host and security product information	WMI `Win32_OperatingSystem` and security-center query in lab only	WMI telemetry, process command lines
Defense evasion simulation	Attempt to invoke AMSI-bypass-like string without disabling AMSI	Echo or write known AMSI bypass tokens as telemetry-only marker	Script block logs, EDR alerts
RAT-like callback	Beacon to internal HTTPS listener	Curl/PowerShell web request with exercise user-agent	DeviceNetworkEvents, proxy, TLS/SNI
Containment	SOC isolates host and removes test artifacts	Cleanup script removes startup file, scheduled task, marker files	IR ticket, endpoint isolation logs

4.3. MITRE ATT&CK mapping

Tactic	Technique	Emulated condition	Data source
Resource Development	T1583.001 Acquire Infrastructure: Domains	Lab domain or subdomain uses aviation-themed naming	DNS registrar inventory, passive DNS
Resource Development	T1583.006 Acquire Infrastructure: Web Services	Internal web service simulates cloud/paste payload staging	Proxy, web server logs
Resource Development	T1608.001 Stage Capabilities: Upload Malware	Benign payload staged on approved internal service	Web logs, file integrity logs
Resource Development	T1588.001 Obtain Capabilities: Malware	Simulate use of commodity RAT with inert tool	Exercise control log
Initial Access	T1566.002 Phishing: Spearphishing Link	Email link to benign VBS or ZIP	Email gateway, URL rewrite logs
Initial Access	T1566.001 Phishing: Spearphishing Attachment	Optional benign attachment variant	Email gateway, sandbox logs
Execution	T1204.001 User Execution: Malicious Link	User clicks approved exercise link	Browser, email click telemetry
Execution	T1204.002 User Execution: Malicious File	User executes benign script	EDR process telemetry
Execution	T1059.005 Command and Scripting Interpreter: Visual Basic	VBS launcher starts PowerShell	Process creation, script logs

Execution	T1059.001 Command and Scripting Interpreter: PowerShell	PowerShell downloads test payload	Script block logging, AMSI
Persistence	T1547.001 Registry Run Keys / Startup Folder	Startup-folder VBS marker	File and registry events
Persistence	T1053.005 Scheduled Task	Exercise scheduled task	Task Scheduler logs, EDR
Defense Evasion	T1027.013 Encrypted/Encoded File	Encoded benign command string	Script block, command-line logs
Defense Evasion	T1685 Disable or Modify Tools	Telemetry-only AMSI-bypass string, no actual modification	AMSI, EDR content detection
Defense Evasion	T1036.005 Masquerading	System-like filenames such as `SystemFramework64Bits.vbs` in user paths	File events, process paths
Discovery	T1082 System Information Discovery	Host information collected	Process command line, WMI logs
Discovery	T1518.001 Security Software Discovery	WMI security product query	WMI activity, EDR telemetry
Discovery	T1016.001 Internet Connection Discovery	Connectivity check to approved sink	DNS/proxy, process telemetry
Command and Control	T1568 Dynamic Resolution	Optional lab DDNS-style hostname	DNS logs
Command and Control	T1573.002 Encrypted Channel: Asymmetric Cryptography	HTTPS callback to internal listener	TLS/SNI, proxy, NDR
Command and Control	T1105 Ingress Tool Transfer	Download benign second-stage file	Proxy, endpoint file events
Defense Evasion / Privilege	T1055 Process Injection; T1055.012 Process Hollowing	Simulate only through marker event or benign EDR test harness; do not inject live code	EDR simulation telemetry
Execution	T1218.005 System Binary Proxy Execution: Mshta	Optional controlled mshta launching local benign HTA	Process telemetry
Execution / Management	T1047 Windows Management Instrumentation	WMI queries security software and OS	WMI logs, process telemetry

- 5. Threat Context

AVIATOR SPIDER should be understood as a durable, financially motivated cybercrime cluster rather than a highly tailored espionage operation. The actor's persistence over multiple years, consistent use of aviation and transportation themes, and willingness to rotate commodity RATs make it operationally important despite modest technical sophistication. Public reporting indicates the group has used RAT families including AsyncRAT, NetWire, WSH RAT, Parallax, Agent Tesla, Imminent Monitor, njRAT, Revenge RAT, WarzoneRAT, jRAT, vjw0rm, STRRAT, and others. This breadth supports an analytic model in which the actor buys or reuses available malware and crypters rather than developing a unique implant family.

The intrusion chain usually begins with a believable business request. Proofpoint observed themes such as flight, aircraft, fuel, yacht, charter, cargo, PPE shipment, and aircraft-parts requests. Cisco Talos observed lures that looked like PDF documents

but linked to `.vbs` files hosted on Google Drive. CrowdStrike reports that since 2021 the actor primarily used phishing emails containing links to crypters hosted on Google Drive, and that in early 2025 AVIATOR SPIDER shifted from Google Drive to Coudup as a preferred hosting provider. This hosting behavior matters because defenders often tune detections around attachments while allowing trusted cloud links to pass with reduced scrutiny.

Once the user executes the staged script, public reporting shows a familiar sequence: obfuscated VBS starts PowerShell; PowerShell retrieves additional code or payload from a paste, code-hosting, or cloud-storage platform; the chain performs system and security-product discovery; it may attempt to disable or bypass built-in protections such as AMSI; it installs persistence through startup folders, registry run keys, or scheduled tasks; and it ultimately deploys a commodity RAT. RAT capabilities vary, but common risk outcomes include remote control, keylogging, screenshots, file access, credential theft from browsers, surveillance, and staging for follow-on fraud or sale of access.

Historical infrastructure reporting highlights DDNS and low-cost hosting patterns. Proofpoint documented keywords such as `kimjoy`, `h0pe`, and `grace` in domains and payload staging URLs, with registrars and providers such as Netdorm, No-IP DDNS, xTom GmbH, and Danilenko, Artyom. Cisco Talos connected multiple campaigns to handles and personas such as `akconsult`, `bodmas`, `kimjoy`, `Nassief2018`, and `pablohop`, while explicitly warning that weaker infrastructure overlaps require cautious confidence handling. For defenders, these artifacts are useful training and hunt pivots, but aged dynamic DNS indicators should be treated as context-rich watch signals rather than permanent proof of current compromise.

For the requested purple team exercise, the bottom line is practical: emulate the behavior chain, not the malware. The high-value learning comes from finding whether the enterprise can detect an aviation-themed cloud link, a double-extension script download, script interpreter execution from a user context, PowerShell staging, suspicious persistence in user profile paths, security product discovery, and outbound callbacks to unusual infrastructure. If the SOC can connect those observations quickly, the same playbook will help against AVIATOR SPIDER and many adjacent commodity RAT campaigns.

4.4. Exercise narrative

The recommended scenario is a controlled aviation supplier compromise simulation:

Narrative element	Exercise value
Lure	A procurement or operations user receives a request for aircraft-parts quote details or charter itinerary confirmation
Delivery	The email links to an approved internal web location that mimics a trusted file-sharing page
File	User downloads a benign double-extension script, such as `Aircrafts_PN_ALT_PN_Desc_Qty_Details.pdf.vbs`
Execution	VBS launches PowerShell and writes exercise markers to disk
Staging	PowerShell retrieves a benign payload from an internal paste/web simulator
Persistence	Startup folder and scheduled task markers emulate TA2541 persistence without hiding intent from exercise control
Discovery	WMI and PowerShell collect OS and security-product names, saving output locally for blue-team recovery
C2	Payload beacons to an internal HTTPS listener with an AVIATOR-SPIDER-EXERCISE user-agent
Response	Blue team investigates, scopes, contains, removes persistence, and produces an incident report

4.5. Controls and constraints

Control area	Required guardrail
Legal/HR	Written authorization for any employee-facing lure; otherwise use SOC-visible lab execution only
Payload safety	No credential theft, screenshot capture, keylogging, browser-cookie access, file exfiltration, destructive commands, or live RAT code
Infrastructure	Use internal listener and internal storage; do not abuse public Google Drive, Discord, GitHub, Coudup, PasteText, or ShareText for payload hosting
Endpoint cleanup	Provide deterministic cleanup script before execution begins
Exercise markers	Prefix all artifacts with `LBC_AVIATOR_EXERCISE` or equivalent to separate test artifacts from real incidents
Data handling	Store only hostnames, usernames assigned to test accounts, timestamps, and telemetry needed for validation
Rules of engagement	Define stop conditions, escalation contacts, target hosts, test accounts, and maximum runtime

- 6. Detection and Hunting

4.6. Detection coverage matrix

Detection objective	Telemetry source	Example signal	Priority
Aviation-themed phishing link or attachment	Email gateway, M365 Defender, SEG logs	Subject/body contains aircraft, charter, fuel, cargo, trip itinerary, parts quote plus external file-sharing URL	High
Double-extension script download	Proxy, browser, EDR file events	`pdf.vbs`, `doc.vbs`, `pif`, `vbs` from cloud/paste/download domain	High
VBS launching PowerShell	Endpoint process telemetry	`wscript.exe` or `cscript.exe` parent with `powershell.exe` child	Critical
Encoded or remote PowerShell retrieval	Script block logs, process command line	`-EncodedCommand`, `DownloadString`, `Invoke-WebRequest`, `ExecutionPolicy RemoteSigned`	Critical
User-profile persistence	Registry, file, EDR	VBS/PS1/EXE in Startup folder or HKCU Run pointing to AppData/Temp	Critical
Scheduled task persistence	Windows events, EDR	Task created by script interpreter or pointing to AppData script	High
WMI security product discovery	WMI logs, process telemetry	Queries to SecurityCenter2 / AntiVirusProduct / firewall	Medium-high

		products	
AMSI bypass attempt	Script block, AMSI, EDR	Known AMSI bypass strings, memory patch strings, `amsiInitFailed` markers	High
Dynamic DNS C2 pattern	DNS, proxy, NDR	DDNS domains, suspicious newly seen hostnames, low-reputation infrastructure	Medium-high
RAT-like outbound behavior	Network, EDR, proxy	Repeated HTTPS callbacks from unusual process or script interpreter	High

4.7. Microsoft Defender KQL hunts

Hunt for VBS or script files launched from email/download contexts and spawning PowerShell:

```
DeviceProcessEvents
| where Timestamp > ago(30d)
| where FileName in~ ("wscript.exe", "cscript.exe", "mshta.exe")
| where ProcessCommandLine has_any (".vbs", ".hta", ".js", ".jse", ".vbe")
| join kind=leftouter (
    DeviceProcessEvents
    | where Timestamp > ago(30d)
    | where FileName in~ ("powershell.exe", "pwsh.exe", "cmd.exe")
    | project ChildTimestamp=Timestamp, DeviceId, InitiatingProcessId, ChildFileName=FileName,
ChildCommandLine=ProcessCommandLine
) on DeviceId, $left.ProcessId == $right.InitiatingProcessId
| project Timestamp, DeviceName, AccountName, FileName, ProcessCommandLine, ChildTimestamp,
ChildFileName, ChildCommandLine, InitiatingProcessFileName, ReportId
| order by Timestamp desc
```

Hunt for TA2541-like PowerShell staging from cloud, paste, or code-hosting domains:

```
DeviceProcessEvents
| where Timestamp > ago(30d)
| where FileName in~ ("powershell.exe", "pwsh.exe")
| where ProcessCommandLine has_any ("DownloadString", "Invoke-WebRequest", "iwr", "curl", "Start-
BitsTransfer", "ExecutionPolicy RemoteSigned", "-enc", "-EncodedCommand")
| where ProcessCommandLine has_any ("drive.google", "onedrive", "discord", "paste", "sharetext",
"github", "raw.githubusercontent", "coudup", "ddns", "publicvm", "zaproxy", "linkpc")
| project Timestamp, DeviceName, AccountName, InitiatingProcessFileName, FileName, ProcessCommandLine,
SHA256, ReportId
| order by Timestamp desc
```

Hunt for startup-folder or HKCU Run persistence created by suspicious script chains:

```
let suspiciousParents = dynamic(["wscript.exe", "cscript.exe", "powershell.exe", "pwsh.exe",
"mshta.exe", "cmd.exe"]);
DeviceFileEvents
| where Timestamp > ago(30d)
| where FolderPath has_any ("\\Microsoft\\Windows\\Start Menu\\Programs\\Startup", "\\AppData\\
\\Roaming", "\\AppData\\Local\\Temp")
| where FileName endswith_cs ".vbs" or FileName endswith_cs ".ps1" or FileName endswith_cs ".exe" or
FileName endswith_cs ".txt"
| where InitiatingProcessFileName in~ (suspiciousParents)
| project Timestamp, DeviceName, AccountName, ActionType, FolderPath, FileName,
InitiatingProcessFileName, InitiatingProcessCommandLine, SHA256
| union (
    DeviceRegistryEvents
    | where Timestamp > ago(30d)
    | where RegistryKey has_any ("\\Software\\Microsoft\\Windows\\CurrentVersion\\Run", "\\Software\\
Microsoft\\Windows\\CurrentVersion\\RunOnce")
    | where InitiatingProcessFileName in~ (suspiciousParents)
    | project Timestamp, DeviceName, AccountName, ActionType, FolderPath=RegistryKey,
```

```

FileName=RegistryValueName, InitiatingProcessFileName, InitiatingProcessCommandLine, SHA256=""
)
| order by Timestamp desc

```

Hunt for WMI security-product discovery and internet-connection checks:

```

DeviceProcessEvents
| where Timestamp > ago(30d)
| where ProcessCommandLine has_any ("SecurityCenter2", "AntiVirusProduct", "FirewallProduct",
"Win32_OperatingSystem", "Win32_ComputerSystem", "Get-WmiObject", "Get-CimInstance", "wmic")
or ProcessCommandLine has_any ("ping us.cnn.com", "Test-NetConnection", "Invoke-WebRequest http",
"Invoke-WebRequest https")
| where InitiatingProcessFileName in~ ("powershell.exe", "pwsh.exe", "cmd.exe", "wscript.exe",
"cscript.exe", "mshta.exe")
or FileName in~ ("powershell.exe", "pwsh.exe", "wmic.exe", "ping.exe")
| project Timestamp, DeviceName, AccountName, FileName, ProcessCommandLine, InitiatingProcessFileName,
InitiatingProcessCommandLine, ReportId
| order by Timestamp desc

```

Hunt for suspicious external callbacks from script interpreters or AppData payloads:

```

DeviceNetworkEvents
| where Timestamp > ago(30d)
| where InitiatingProcessFileName in~ ("powershell.exe", "pwsh.exe", "wscript.exe", "cscript.exe",
"mshta.exe", "regsvcs.exe", "msbuild.exe", "installutil.exe")
or InitiatingProcessFolderPath has_any ("\\AppData\\Roaming", "\\AppData\\Local\\Temp", "\\Users\\
\\Public")
| where RemoteUrl has_any ("ddns", "publicvm", "zapto", "linkpc", "duckdns", "paste", "sharetext",
"github", "discord", "drive.google", "onedrive", "coudup")
or RemotePort in (443, 80, 8080, 8443)
| project Timestamp, DeviceName, AccountName, InitiatingProcessFileName, InitiatingProcessCommandLine,
RemoteUrl, RemoteIP, RemotePort, Protocol, ReportId
| order by Timestamp desc

```

4.8. Splunk SPL hunts

Search for VBS-to-PowerShell process chains:

```

index=edr sourcetype=process earliest=-30d
(Image="*\\wscript.exe" OR Image="*\\cscript.exe" OR Image="*\\mshta.exe")
(CommandLine="*.vbs*" OR CommandLine="*.hta*" OR CommandLine="*.vbe*")
| eval parent_pid=ProcessId, host_key=coalesce(DeviceId, host)
| join type=left host_key parent_pid [
search index=edr sourcetype=process earliest=-30d (Image="*\\powershell.exe" OR Image="*\\pwsh.exe"
OR Image="*\\cmd.exe")
| eval host_key=coalesce(DeviceId, host), parent_pid=ParentProcessId
| fields _time host_key parent_pid Image CommandLine ParentImage
]
| table _time host user Image CommandLine ParentImage

```

Search for suspicious persistence under user profile paths:

```

(index=edr sourcetype=file earliest=-30d (file_path="*\\Microsoft\\Windows\\Start Menu\\Programs\\
\\Startup\\" OR file_path="*\\AppData\\Roaming\\" OR file_path="*\\AppData\\Local\\Temp\\")
(file_name="*.vbs" OR file_name="*.ps1" OR file_name="*.exe" OR file_name="*.txt"))
OR (index=edr sourcetype=registry earliest=-30d registry_path="*\\Software\\Microsoft\\Windows\\
\\CurrentVersion\\Run*")
| search process_name IN
("wscript.exe", "cscript.exe", "powershell.exe", "pwsh.exe", "mshta.exe", "cmd.exe")
| table _time host user action file_path registry_path registry_value process_name process_command_line

```

Search for TA2541-like staging and DDNS/cloud hosting:

```

index=proxy OR index=dns OR index=edr earliest=-30d
("drive.google" OR "onedrive" OR "discord" OR "paste.ee" OR "pastetext" OR "sharetext" OR "github" OR
"raw.githubusercontent" OR "coudup" OR "ddns" OR "publicvm" OR "zapto" OR "linkpc" OR "duckdns")
| eval indicator=coalesce(url, query, dest_host, RemoteUrl, CommandLine)
| search indicator="*.vbs*" OR indicator="*.ps1*" OR indicator="*.txt*" OR indicator="*.exe*" OR
indicator="*DownloadString*" OR indicator="*Invoke-WebRequest*" OR indicator="*EncodedCommand*"
| table _time host user src_ip dest_ip dest_host url process_name process_command_line indicator

```

4.9. Sigma detection logic

```

title: Script Interpreter Launching PowerShell From TA2541-Like Delivery Chain
id: 8e5e09af-aviator-spider-emulation-001
status: test
description: Detects VBS/MSHTA script interpreter execution that spawns PowerShell with staging-related
command-line patterns consistent with TA2541-like delivery chains.
references:
  - https://attack.mitre.org/groups/G1018/
  - https://www.proofpoint.com/us/blog/threat-insight/charting-ta2541s-flight
author: Lost Boys Cyber
date: 2026/06/24
tags:
  - attack.execution
  - attack.t1059.001
  - attack.t1059.005
  - attack.t1204
  - attack.t1105
logsource:
  product: windows
  category: process_creation
detection:
  selection_parent:
    ParentImage|endswith:
      - '\\wscript.exe'
      - '\\cscript.exe'
      - '\\mshta.exe'
  selection_child:
    Image|endswith:
      - '\\powershell.exe'
      - '\\pwsh.exe'
  selection_staging:
    CommandLine|contains:
      - 'DownloadString'
      - 'Invoke-WebRequest'
      - 'iwr '
      - 'Start-BitsTransfer'
      - 'ExecutionPolicy RemoteSigned'
      - '-EncodedCommand'
      - 'drive.google'
      - 'discord'
      - 'github'
      - 'paste'
      - 'coudup'
  condition: selection_parent and selection_child and selection_staging
falsepositives:
  - Administrative scripts launched from known signed management tooling
level: high

```

```

title: User Profile Startup Persistence From Script Interpreter
id: b37f5e13-aviator-spider-emulation-002
status: test
description: Detects script or executable persistence written to Startup folder or HKCU Run by script
interpreters, matching common TA2541 persistence patterns.
references:
  - https://attack.mitre.org/groups/G1018/
  - https://www.proofpoint.com/us/blog/threat-insight/charting-ta2541s-flight
author: Lost Boys Cyber
date: 2026/06/24
tags:
  - attack.persistence
  - attack.t1547.001
  - attack.t1053.005
logsource:
  product: windows
  category: file_event
detection:

```

```

selection_path:
  TargetFilename|contains:
    - '\\Microsoft\\Windows\\Start Menu\\Programs\\Startup\\'
    - '\\AppData\\Roaming\\'
    - '\\AppData\\Local\\Temp\\'
selection_extension:
  TargetFilename|endswith:
    - '.vbs'
    - '.ps1'
    - '.exe'
    - '.txt'
selection_process:
  Image|endswith:
    - '\\wscript.exe'
    - '\\cscript.exe'
    - '\\powershell.exe'
    - '\\pwsh.exe'
    - '\\mshta.exe'
    - '\\cmd.exe'
condition: selection_path and selection_extension and selection_process
falsepositives:
  - Logon scripts deployed by enterprise software distribution; validate signer and path
level: high

```

4.10. YARA-style content triage for downloaded scripts

This rule is intended for file triage or sandbox pre-filtering of downloaded scripts. Validate and tune before production use.

```

rule LBC_TA2541_Like_VBS_PowerShell_Stager
{
  meta:
    description = "Detects VBS launchers with PowerShell staging patterns similar to TA2541"
  reporting:
    author = "Lost Boys Cyber"
    date = "2026-06-24"
    reference = "https://attack.mitre.org/groups/G1018/"
    tlp = "TLP:CLEAR"
  strings:
    $vbs_shell = "CreateObject(\"WScript.Shell\")" nocase
    $run = ".Run" nocase
    $powershell = "PowerShell" nocase
    $remote_signed = "ExecutionPolicy RemoteSigned" nocase
    $download_1 = "DownloadString" nocase
    $download_2 = "Invoke-WebRequest" nocase
    $strrev = "StrReverse" nocase
    $startup = "Start Menu\\Programs\\Startup" nocase
    $cloud_1 = "drive.google" nocase
    $cloud_2 = "paste" nocase
    $cloud_3 = "github" nocase
  condition:
    uint16(0) != 0x5A4D and
    $vbs_shell and $run and $powershell and
    1 of ($remote_signed, $download_*) and
    1 of ($strrev, $startup, $cloud_*)
}

```

4.11. Purple team injects and expected blue-team observations

Inject	Red-team action	Expected blue-team signal	Expected analyst decision
1	Send approved aviation-themed test email with cloud-link mimic	Email alert or URL-click telemetry	Identify sector-specific lure and external file-sharing risk
2	Download benign double-	Proxy/file event for VBS from	Quarantine or isolate test file;

	extension `pdf.vbs`	web	investigate recipient scope
3	Execute VBS on lab host	`wscript.exe` to `powershell.exe` process chain	Escalate as malicious script execution
4	PowerShell retrieves benign text payload	Network event and PowerShell script block	Determine source URL, downloaded file, and parent process
5	Create startup-folder VBS marker and scheduled task	Persistence alerts	Contain host and preserve artifacts before cleanup
6	Run WMI security-product discovery	WMI/process alerts	Treat as reconnaissance supporting malware staging
7	Beacon to internal HTTPS listener	Repeated callback from suspicious process	Block domain, isolate host, scope other devices
8	Cleanup and after-action review	Artifact removal and no residual persistence	Validate response completeness and detection gaps

The following scripts are safe, benign emulation aids for the injects above. They create marker artifacts, exercise telemetry paths, and call back only to an internally controlled listener. Operators should run them only on approved lab endpoints under a written rules-of-engagement window, never with production credentials, and never against third-party infrastructure.

Inject 1 support: internal landing page for the approved aviation-themed lure. Host this only on an internal exercise web server and link to a benign marker file rather than a weaponized attachment.

```
<!doctype html>
<html lang="en">
<head>
  <meta charset="utf-8">
  <title>Approved Aviation Vendor Portal Exercise</title>
</head>
<body>
  <h1>Lost Boys Cyber Purple Team Exercise</h1>
  <p>This is an authorized training page for AVIATOR SPIDER-style cloud-link telemetry validation.</p>
  <p>Exercise marker: LBC-PTE-AVIATOR-SPIDER-001</p>
  <a href="/downloads/Aviation_Manifest_Review.pdf.vbs.txt">Download benign marker file</a>
</body>
</html>
```

Injects 2-6 support: benign Windows endpoint emulator. Save as `Invoke-LBC-AviatorSpiderEmulation.ps1` and execute from an approved lab host. The script writes marker files, records an event-log marker where permitted, launches a harmless child PowerShell process, creates then removes a startup-folder marker, creates then removes a scheduled-task marker, and runs read-only WMI discovery for telemetry. It does not disable security tooling, steal credentials, inject code, or deploy malware.

```
<#
.SYNOPSIS
  Benign AVIATOR SPIDER-style purple-team telemetry emulator.
.SAFETY
  Authorized lab use only. No credential access, no security bypass, no malware, no code injection.
#>
param(
  [string]$ExerciseId = "LBC-PTE-AVIATOR-SPIDER-001",
  [string]$InternalListener = "https://pte-listener.internal.example.test/collect",
  [string]$WorkDir = "$env:TEMP\LBC-PTE-AviatorSpider"
)

$ErrorActionPreference = "Stop"
New-Item -ItemType Directory -Force -Path $WorkDir | Out-Null
```

```

$markerPath = Join-Path $WorkDir "Aviation_Manifest_Review.pdf.vbs.txt"
$payloadPath = Join-Path $WorkDir "benign-stage-two.txt"

"$ExerciseId benign VBS/cloud-download marker $(Get-Date -Format o)" | Set-Content -Path $markerPath
-Encoding UTF8
"$ExerciseId benign stage-two marker $(Get-Date -Format o)" | Set-Content -Path $payloadPath -Encoding
UTF8

try {
    New-EventLog -LogName Application -Source "LBC-PurpleTeam" -ErrorAction SilentlyContinue
    Write-EventLog -LogName Application -Source "LBC-PurpleTeam" -EntryType Information -EventId 41001
    -Message "$ExerciseId started benign AVIATOR SPIDER emulation"
} catch {
    Write-Host "Event-log marker skipped: $($_.Exception.Message)"
}

# Inject 3: generate script-interpreter parent/child telemetry without harmful logic.
Start-Process -FilePath "powershell.exe" -ArgumentList @(
    "-NoProfile",
    "-ExecutionPolicy", "RemoteSigned",
    "-Command", "Write-Output '$ExerciseId benign child PowerShell execution'; Start-Sleep -Seconds 2"
) -WindowStyle Hidden -Wait

# Inject 4: simulate staged retrieval using an internal listener if reachable; otherwise preserve a
local marker.
try {
    Invoke-WebRequest -Uri "$InternalListener/stage-two.txt?exercise=$ExerciseId" -UseBasicParsing
    -TimeoutSec 5 -OutFile $payloadPath
} catch {
    "Listener unavailable; retained local benign stage-two marker. $($_.Exception.Message)" | Add-Content
    -Path $payloadPath
}

# Inject 5: create persistence-shaped markers, then remove them during cleanup.
$startupMarker = Join-Path ([Environment]::GetFolderPath("Startup")) "LBC-PTE-AviatorSpider-Marker.vbs"
"" $ExerciseId benign startup marker; no execution logic" | Set-Content -Path $startupMarker -Encoding
ASCII
$taskName = "LBC-PTE-AviatorSpider-Marker"
schtasks.exe /Create /TN $taskName /SC ONCE /ST 23:59 /TR "cmd.exe /c echo $ExerciseId benign scheduled
task marker" /F | Out-Null

# Inject 6: read-only discovery telemetry.
Get-CimInstance -ClassName Win32_OperatingSystem | Select-Object Caption, Version, BuildNumber | Out-
File (Join-Path $WorkDir "os-discovery.txt")
Get-CimInstance -Namespace root/SecurityCenter2 -ClassName AntivirusProduct -ErrorAction
SilentlyContinue |
    Select-Object displayName, productState, pathToSignedProductExe |
    Out-File (Join-Path $WorkDir "security-product-discovery.txt")

Write-Output "Created benign exercise markers under $WorkDir"
Write-Output "Startup marker: $startupMarker"
Write-Output "Scheduled task marker: $taskName"

```

Inject 7 support: internal HTTPS listener for beacon telemetry. Run this only inside the exercise network and replace the example certificate paths with approved lab certificates.

```

#!/usr/bin/env python3
"""Benign purple-team callback collector for AVIATOR SPIDER-style telemetry exercises."""
from __future__ import annotations

import datetime as dt

```



```

import http.server
import json
import ssl
from pathlib import Path
from urllib.parse import parse_qs, urlparse

HOST = "0.0.0.0"
PORT = 8443
LOG_PATH = Path("lbc-pte-aviator-spider-callbacks.jsonl")
CERT_FILE = "lab-listener.crt"
KEY_FILE = "lab-listener.key"

class Handler(http.server.BaseHTTPRequestHandler):
    def do_GET(self) -> None:
        parsed = urlparse(self.path)
        record = {
            "timestamp": dt.datetime.utcnow().isoformat(timespec="seconds") + "Z",
            "client": self.client_address[0],
            "path": parsed.path,
            "query": parse_qs(parsed.query),
            "user_agent": self.headers.get("User-Agent", ""),
            "exercise_marker": "LBC-PTE-AVIATOR-SPIDER-001",
        }
        with LOG_PATH.open("a", encoding="utf-8") as fh:
            fh.write(json.dumps(record, sort_keys=True) + "\n")
        body = b"LBC benign callback received\n"
        self.send_response(200)
        self.send_header("Content-Type", "text/plain")
        self.send_header("Content-Length", str(len(body)))
        self.end_headers()
        self.wfile.write(body)

    def log_message(self, fmt: str, *args) -> None:
        print(f"{self.address_string()} - {fmt % args}")

if __name__ == "__main__":
    server = http.server.ThreadingHTTPServer((HOST, PORT), Handler)
    context = ssl.SSLContext(ssl.PROTOCOL_TLS_SERVER)
    context.load_cert_chain(certfile=CERT_FILE, keyfile=KEY_FILE)
    server.socket = context.wrap_socket(server.socket, server_side=True)
    print(f"Listening on https://{HOST}:{PORT}; writing {LOG_PATH}")
    server.serve_forever()

```

Inject 8 support: cleanup script for the lab host after observations are captured.

```

param(
    [string]$WorkDir = "$env:TEMP\LBC-PTE-AviatorSpider",
    [string]$TaskName = "LBC-PTE-AviatorSpider-Marker"
)

$startupMarker = Join-Path ([Environment]::GetFolderPath("Startup")) "LBC-PTE-AviatorSpider-Marker.vbs"
Remove-Item -Path $startupMarker -Force -ErrorAction SilentlyContinue
schtasks.exe /Delete /TN $TaskName /F 2>$null | Out-Null
Remove-Item -Path $WorkDir -Recurse -Force -ErrorAction SilentlyContinue
Write-Output "Removed benign AVIATOR SPIDER exercise markers. Validate EDR/SIEM closure evidence before closing the exercise."

```

• 7. Recommendations

Priority	Recommendation	Owner	Rationale	Validation method
1	Block or warn on script downloads from file-sharing, paste, and	Email/Web security	TA2541 repeatedly abuses trusted hosting	Attempt exercise download and verify

	newly seen domains		and cloud links	block/logging
1	Alert on `wscript.exe`, `cscript.exe`, or `mshta.exe` launching PowerShell	SOC / EDR	This is a high-confidence execution chain for TA2541-like delivery	Run benign emulator and confirm alert fidelity
1	Enable PowerShell Script Block Logging, Module Logging, and command-line capture	Endpoint engineering	Staging and AMSI-bypass attempts require script visibility	Confirm query returns full commands and decoded script content
1	Monitor startup folder, HKCU Run, and scheduled task creation from script interpreters	SOC / EDR	TA2541 commonly uses these persistence methods	Run persistence inject and verify alert plus artifact path
2	Build aviation-lure email detections and user-reporting playbooks	Messaging / Awareness	Sector-specific lures are stable and realistic	Send approved test lures and measure reporting/triage
2	Add DDNS/cloud/paste infrastructure watchlists with context scoring	SOC / Threat Intel	Infrastructure patterns are useful but can create false positives	Tune against 30 days of proxy/DNS logs before blocking
2	Add enrichment fields for URL, file extension, referrer, parent process, and user to detections	SIEM engineering	Analysts need chain context to distinguish TA2541-like intrusions from benign admin activity	Review alert payload completeness during exercise
2	Create an incident-response checklist for commodity RAT phishing	IR lead	The likely impact includes remote access and credential exposure	Tabletop containment and scoping decisions after inject 7
3	Maintain historical TA2541 IOCs as watch/context, not blanket block-only indicators	Threat Intel	Aged DDNS and dynamic infrastructure may be reassigned or stale	Label IOCs by source date and actionability
3	Run quarterly purple team replay with altered hosting and lure variants	Purple team lead	Actor tradecraft changes incrementally; detections should generalize	Repeat with Google Drive mimic, Coudup mimic, GitHub mimic, and attachment variant

Immediate control checks:

- Confirm all Windows endpoints record process command line, parent process, SHA256, file path, user, and network destination.
- Confirm the SIEM can join email-click, proxy, endpoint-process, file, registry, and DNS events by user/device/time window.
- Confirm endpoint policy blocks or warns on VBS from internet-origin locations where business-acceptable.
- Confirm AMSI is enabled and tamper protection cannot be disabled by standard users.
- Confirm EDR alerts include enough context to distinguish an exercise marker from a real event without suppressing the underlying behavior.

Containment playbook for a real TA2541-like detection:

Step	Action	Evidence to preserve
1	Isolate suspected host or apply network containment	Hostname, user, isolation timestamp, EDR case ID
2	Preserve process tree and script content	VBS/PS1 content, PowerShell logs, AMSI hits, command lines
3	Identify delivery source and recipients	Email message ID, sender, URL, attachment hash, all recipients/clickers
4	Remove persistence	Startup folder files, scheduled tasks, HKCU Run values
5	Scope network callbacks	DNS/proxy/EDR connections to staging and C2 domains
6	Review credential exposure	Browser credential stores, identity logs, impossible travel, MFA changes, mailbox rules
7	Block indicators and behaviors	URLs, domains, hashes, detection rules, mail purge
8	Report and improve	Root cause, missed controls, detection changes, awareness follow-up

- 8. References
- [1] CrowdStrike, “Aviator Spider Adversary Profile,” accessed 2026-06-24, <https://www.crowdstrike.com/en-us/adversaries/aviator-spider/>
- [2] MITRE ATT&CK, “TA2541, Group G1018,” version 1.1, last modified 2024-04-10, accessed 2026-06-24, <https://attack.mitre.org/groups/G1018/>
- [3] Proofpoint, Selena Larson and Joe Wise, “Charting TA2541's Flight,” 2022-02-15, <https://www.proofpoint.com/us/blog/threat-insight/charting-ta2541s-flight>
- [4] Cisco Talos, Vitor Ventura / Tiago Pereira, “Operation Layover: How we tracked an attack on the aviation industry to five years of compromise,” 2021-09-16, <https://blog.talosintelligence.com/operation-layover-how-we-tracked-attack/>
- [5] MITRE ATT&CK, “AsyncRAT, Software S1087,” accessed 2026-06-24, <https://attack.mitre.org/software/S1087/>
- [6] MITRE ATT&CK, “Snip3, Software S1086,” accessed 2026-06-24, <https://attack.mitre.org/software/S1086/>
- [7] Proofpoint / Emerging Threats signatures cited in Proofpoint TA2541 reporting: ET POLICY Pastebin-style Service in TLS SNI; ET HUNTING PowerShell request for paste.ee page; ET MALWARE PowerShell with decimal encoded RUNPE downloaded; ETPRO hunting Google Drive double-extension VBS/PIF downloads.